

Projet: Reconnaissance de caractères grâce à l'intelligence artificielle

Table des matières

I - Exposition du problème	2
II - Etude n°1: Projet commun	3
1. Principe	3
2. Réalisation du réseau	4
3. Analyse des résultats	5
III - Etude n°2: Prédiction par méthode ensembliste	8
1. Réalisation des trois réseaux	8
2. Méthode des moyennes de la sortie	8
3. Méthode des occurrences	9
a. Principe général	9
b. Moyenne quand les réseaux sont pas d'accord	11
c. Maximum quand les réseaux sont pas d'accord	11
4. Analyse des résultats	12
IV - Conclusion	14
1. Conclusion sur la meilleure méthode	14
1. Potentiels approfondissements	15
a. Augmenter le nombre de réseaux	15
b. Pondérer chacune des moyennes par la précision de chaque réseau	16
c. Concevoir, entraîner et tester un réseau spécialisé	17
3. Point de vue personnel	17
Annexe	18

I - Exposition du problème

Nous allons chercher à faire un algorithme de reconnaissance automatique de nombres en classification par apprentissage automatique (Machine Learning). Nous effectuerons cela en utilisant les images d'une database MNIST de dimension 28×28 . Les couleurs du chiffre et du fond seront tirées aléatoirement, elles sont représentées par un vecteur de format (R,G,B). Notre problème est donc un problème à 10 classes.

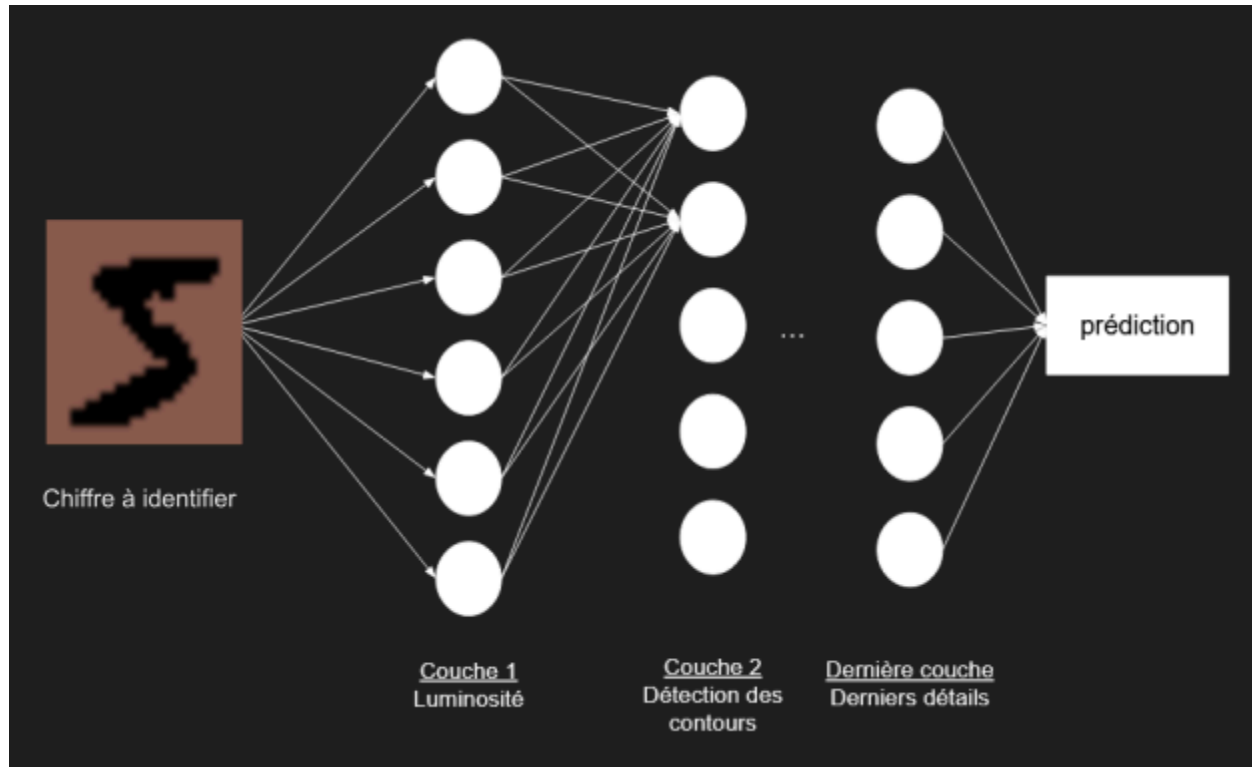
Dans un premier temps, nous chercherons à créer, entraîner et tester un réseau de neurones pour résoudre ce problème. Ce problème comportant peu de classes, nous essaierons différents réseaux de façon à obtenir une précision supérieure à 97%.

Ensuite nous essaierons d'améliorer la précision de la prédiction en utilisant 3 réseaux différents avec agrégation de prédictions ce qui nous permettrait d'avoir des réseaux qui se focalisent sur des aspects différents et ainsi, d'améliorer la précision. La difficulté majeure sera de choisir la prédiction lorsque les trois réseaux sont en désaccord. On verra ainsi différentes façons d'établir les prédictions.

II - Etude n°1: Projet commun

1. Principe

Le but de cette partie du projet est de déterminer quel chiffre est contenu dans l'image à l'entrée de notre réseau de neurones. Le réseau de neurones est constitué d'un ensemble de couches de convolution mises les unes à la suite des autres, chacune de ces couches est chargée de la détection de détails de plus en plus fins: luminosité, contours etc... , à la manière d'un cortex visuel humain.



La prédiction attendue ici est un vecteur 1×10 contenant l'indice de confiance pour chaque chiffre (ex: [2.4900142e-08 1.7638170e-11 1.0000000e+00 3.0744927e-12 1.2482224e-10 1.6179590e-14 1.9282476e-09 8.3443573e-12 3.1442254e-10 1.6870097e-10] ici le réseau prédit 2).

Il faut donc procéder à une réduction de la dimension des éléments envoyés en entrée, à savoir les images qui sont des vecteurs de dimension $28 \times 28 \times 3$. Pour cela, on utilise des couches de Maxpooling avec un noyau de (2,2) et une couche dense à la fin.

L'architecture en elle-même est le fruit de plusieurs essais en faisant varier différents paramètres comme le nombre de couches de convolution 2D, le nombre de neurones par couches et les fonctions d'activation (ReLU, Leaky_ReLu).

En sortie du réseau, la présence de deux couches denses était fortement conseillée, c'est ce qu'on a donc utilisé.

L'entraînement de ce réseau à l'aide d'une base de données constituée d'images de chiffre et de leur valeur permet le bon réglage des poids du réseau. 80 % des données sont utilisées pour l'entraînement, 20% pour le test et la validation.

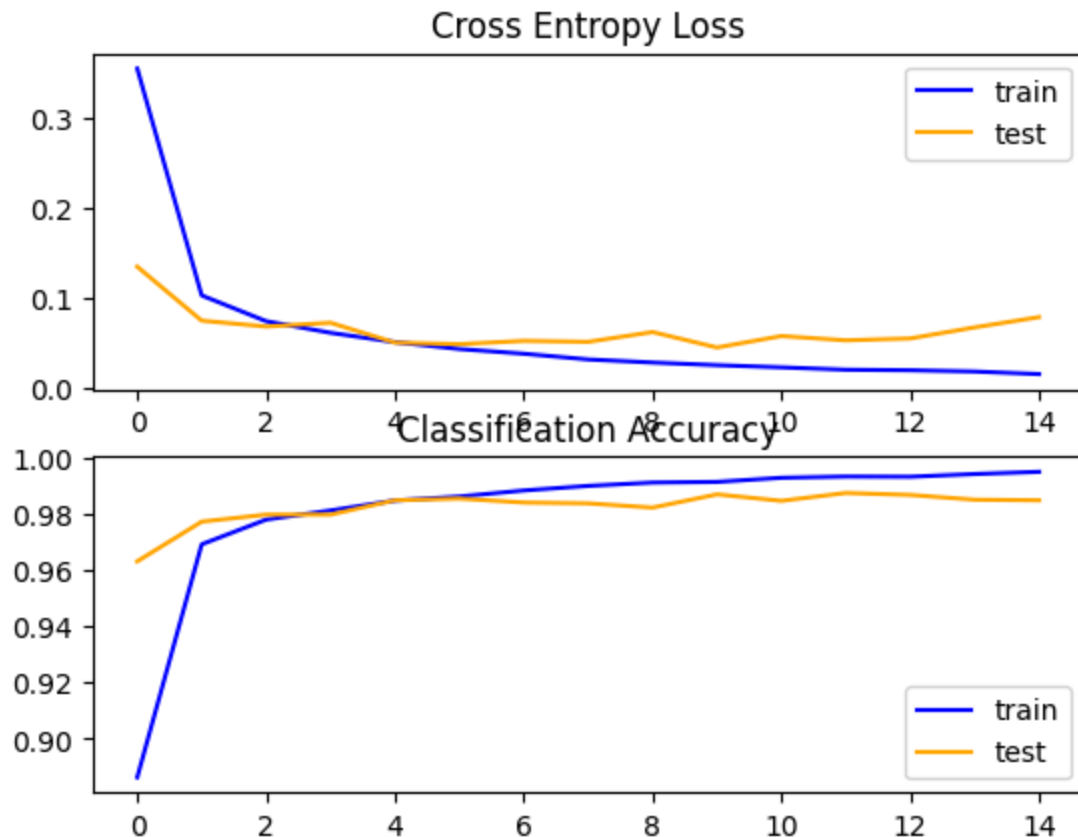
2. Réalisation du réseau

Voici l'architecture finale:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 512)	14336
conv2d_1 (Conv2D)	(None, 24, 24, 256)	1179904
conv2d_2 (Conv2D)	(None, 22, 22, 128)	295040
max_pooling2d (MaxPooling2D)	(None, 11, 11, 128)	0
conv2d_3 (Conv2D)	(None, 9, 9, 64)	73792
conv2d_4 (Conv2D)	(None, 7, 7, 32)	18464
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 32)	0
flatten (Flatten)	(None, 288)	0
dense (Dense)	(None, 64)	18496
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 10)	330

=====
 Total params: 1602442 (6.11 MB)
 Trainable params: 1602442 (6.11 MB)
 Non-trainable params: 0 (0.00 Byte)



On constate ici que la courbe de la perte en fonction du nombre d'époques est décroissante et que celle de l'accuracy est croissante. Le réseau converge donc, il est adapté au problème et permet sa résolution.

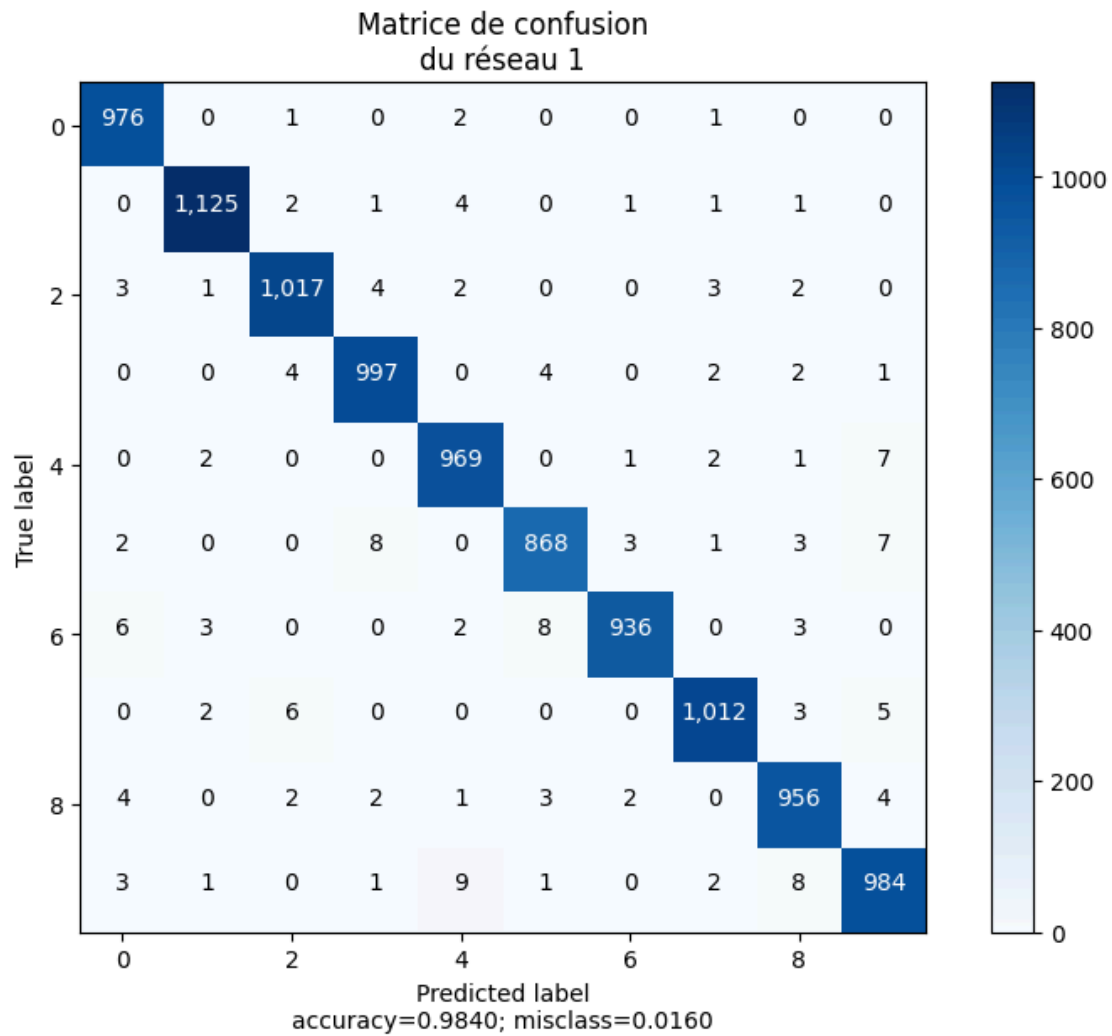
Après un entraînement de 15 epochs avec un batch size de 64, on obtient une accuracy de 98.48%, ce qui est déjà très satisfaisant. Ce score peut être amélioré en effectuant plus d'entraînements mais il ne faut pas en abuser car cela peut conduire à un overfitting du réseau sur un type de données le rendant inutilisable sur d'autres données.

On verra cependant que cela peut encore être amélioré par la méthode utilisée dans la suite.

3. Analyse des résultats

Après la création du réseau de neurones et son entraînement, vient la phase d'inférence autrement dit le test du réseau sur des images qu'il n'a jamais vues.

Le résultat de ce test peut être visualisé clairement à l'aide d'une matrice de confusion:



Ici, la matrice de confusion permet bien de confirmer que l'accuracy de notre réseau est bonne ce qui est matérialisé par des coefficients hors de la diagonale proche de zéro (143 mauvaises réponses sur 10000).

Il est tout de même intéressant de s'intéresser aux images qui sont sources d'erreur du réseau pour comprendre pourquoi elles le sont.

Voici donc toutes les mauvaise réponses:



Deux principales sources d'erreurs peuvent être identifiées:

- Le réseau confond un chiffre avec un autre chiffre qui peut lui ressembler comme le 1 et le 7 ou le 0 et le 6

P=1,T=7



P=0,T=6



- Les chiffres ne sont simplement pas identifiables

P=2,T=6



P=1,T=8



Une solution à ce problème peut être d'avoir recours à plusieurs réseaux de neurones plus ou moins différents qui se complètent. C'est ce que nous avons exploré dans l'étude 2.

III - Etude n°2: Prédiction par méthode ensembliste

1. Réalisation des trois réseaux

Nous allons donc réaliser trois réseaux de façon similaire à l'étude n°1 en faisant de légères variations entre chaque réseau pour bénéficier de potentielles complémentarités de chaque réseau. Nous utiliserons le réseau de la première partie et ces deux réseaux :

Réseau 2		
Layer (type)	Output Shape	Param #
conv2d_5 (Conv2D)	(None, 26, 26, 128)	3584
max_pooling2d_2 (MaxPoolin g2D)	(None, 13, 13, 128)	0
conv2d_6 (Conv2D)	(None, 11, 11, 64)	73792
conv2d_7 (Conv2D)	(None, 9, 9, 32)	18464
max_pooling2d_3 (MaxPoolin g2D)	(None, 4, 4, 32)	0
flatten_1 (Flatten)	(None, 512)	0
dense_3 (Dense)	(None, 32)	16416
dense_4 (Dense)	(None, 10)	330
=====		
Total params: 112586 (439.79 KB)		
Trainable params: 112586 (439.79 KB)		
Non-trainable params: 0 (0.00 Byte)		

Réseau 3		
Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 26, 26, 256)	7168
conv2d_9 (Conv2D)	(None, 24, 24, 128)	295040
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 128)	0
conv2d_10 (Conv2D)	(None, 10, 10, 64)	73792
conv2d_11 (Conv2D)	(None, 8, 8, 32)	18464
max_pooling2d_5 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_5 (Dense)	(None, 64)	131136
dense_6 (Dense)	(None, 32)	2080
dense_7 (Dense)	(None, 10)	330
=====		
Total params: 528010 (2.01 MB)		
Trainable params: 528010 (2.01 MB)		
Non-trainable params: 0 (0.00 Byte)		

2. Méthode des moyennes de la sortie

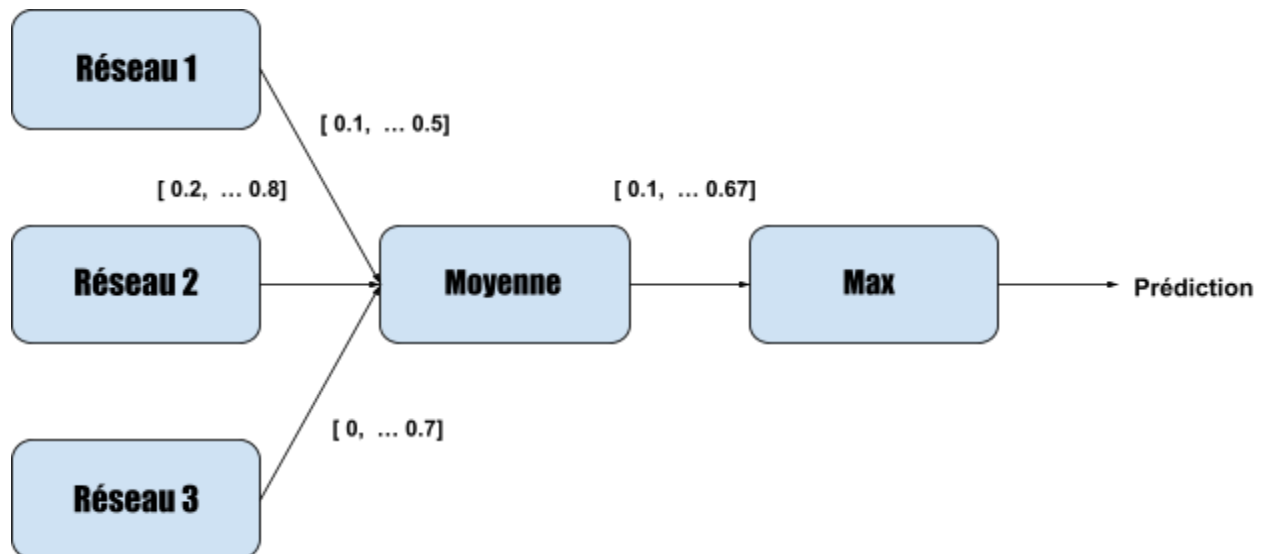
Tout d'abord, nous avons constaté que chaque réseau renvoie une liste `y_pred` qui est un score entre 0 et 1

pour chaque chiffre avant de renvoyer la prédiction avec `Y_pred`.

Ainsi, notre première approche a été de réaliser une moyenne des sorties y_{pred} et de prendre le maximum de chaque moyenne comme la prédiction sans passer par le nombre d'occurrences. C'est cette

méthode ainsi que le choix de la fonction de décision dans la méthode des occurrences qui constitueront notre approfondissement.

Voici le principe:



Nous avons implémenter cela grâce à ce code:

```

for i in range(len(y_pred1)):
    for j in range(10):
        y_pred_f[i][j] = [(y_pred1[i][j]+y_pred2[i][j]+y_pred3[i][j])/3]
Y_pred_f_moy = np.argmax(y_pred_f, 1)
  
```

y_{pred1} , y_{pred2} et y_{pred3} sont les sorties des 3 réseaux.

3. Méthode des occurrences

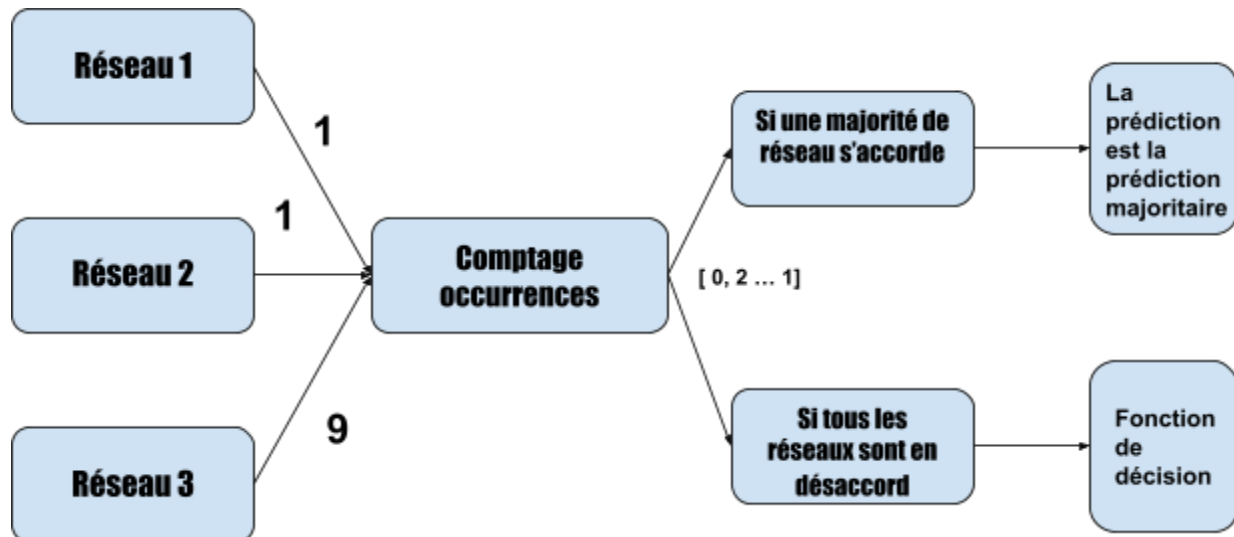
a. Principe général

Cette méthode consiste à compter le nombre d'occurrences de chaque nombre dans les prédictions 3 cas se présentent alors:

- Les trois réseaux sont d'accords

- Deux réseaux s'accordent
- Aucun des réseaux ne fait la même prédiction

Dans les deux premiers cas, on choisit comme prédiction celle avec le plus d'occurrences. C'est dans le 3ème qu'il faut réfléchir à une fonction de décision.



Implémenté dans Python:

```

Occ = []
for j in Agregat:
    Occ.append(OccurrencesClasses ( j , 10 )) # utilisation de la fonction fournie
#On réalise l'algorithme de décision en fonction du nombre d'occurrences
occ1 = []
occ2 = []
occ3 = []
for i in range(len(Occ)):
    if (Occ[i][np.argmax(Occ[i])] == 3):
        occ3.append((i,np.argmax(Occ[i])))
    elif (Occ[i][np.argmax(Occ[i])] == 2):
        occ2.append((i,np.argmax(Occ[i])))
    else:
        occ1.append((i,np.argmax(Occ[i])))

# on considère la valeur bonne si au moins 2 réseaux sont d'accords
Y_pred_f = [0 for i in range(len(y_pred_f))]
for i in range(len(occ2)):
    Y_pred_f[occ2[i][0]] = [occ2[i][1]]
for i in range(len(occ3)):
    Y_pred_f[occ3[i][0]] = [occ3[i][1]]
for i in range(len(occ1)):
    Y_pred_f_doute[occ1[i][0]] = Fonction_de_décision (y_pred1,y_pred2,y_pred3)
  
```

b. Moyenne quand les réseaux sont pas d'accord

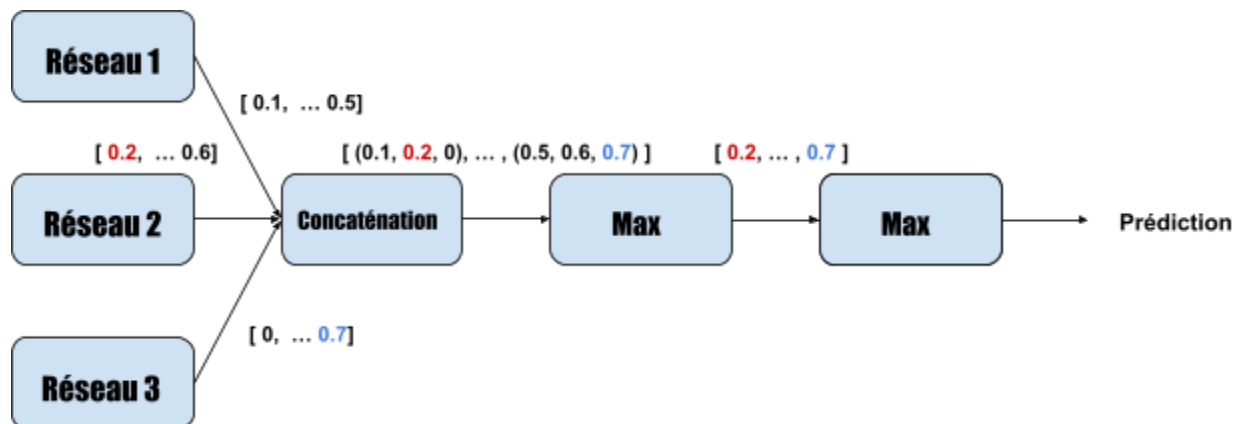
On utilise une fonction similaire à celle abordée dans le 2., uniquement quand les réseaux sont en désaccord.

c. Maximum quand les réseaux sont pas d'accord

Quand les 3 réseaux sont en désaccord, on concatène chaque sortie de réseaux sous une liste de la forme:

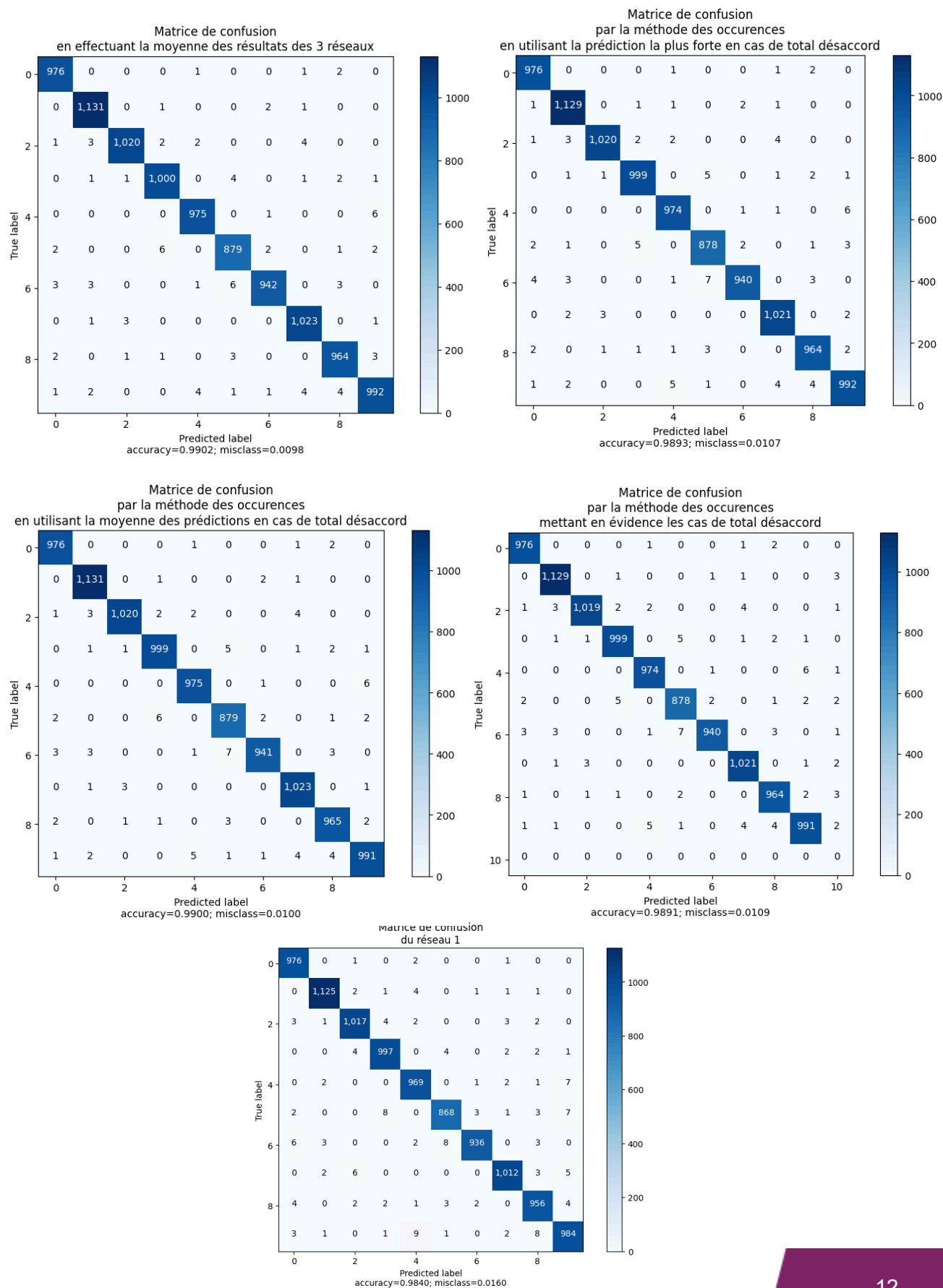
```
[(score_réseau1_pour_0, score_réseau2_pour_0, score_réseau3_pour_0), ..., (score_réseau1_pour_9, score_réseau2_pour_9, score_réseau3_pour_9)]
```

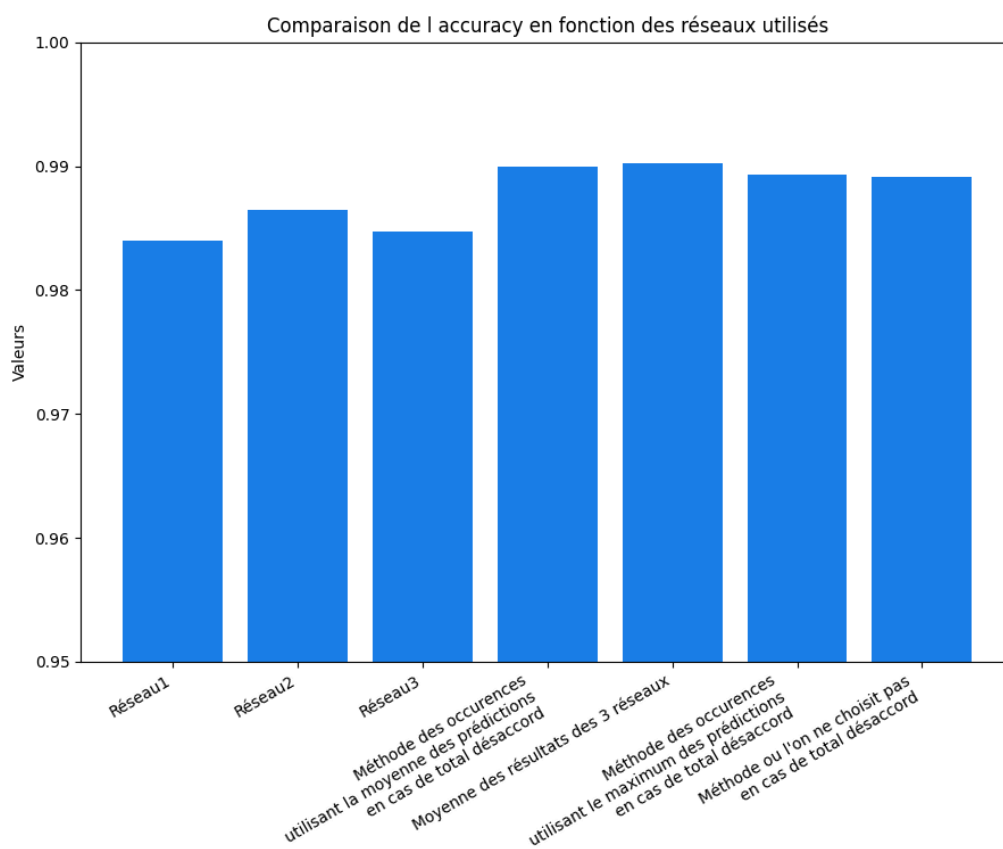
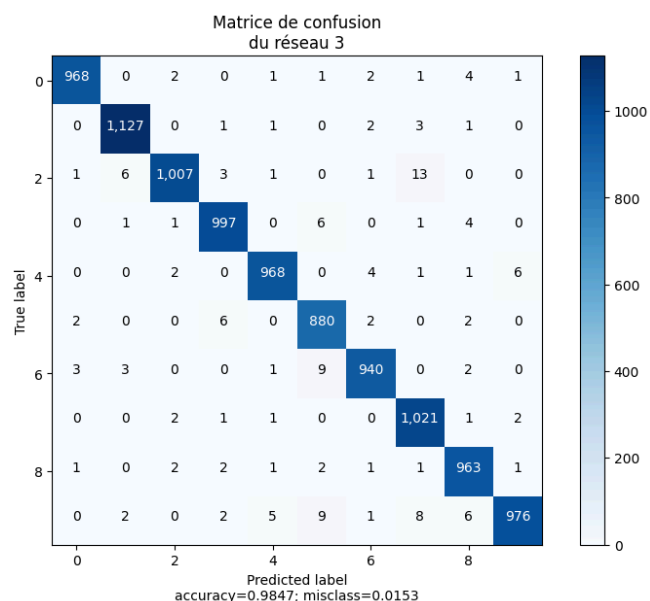
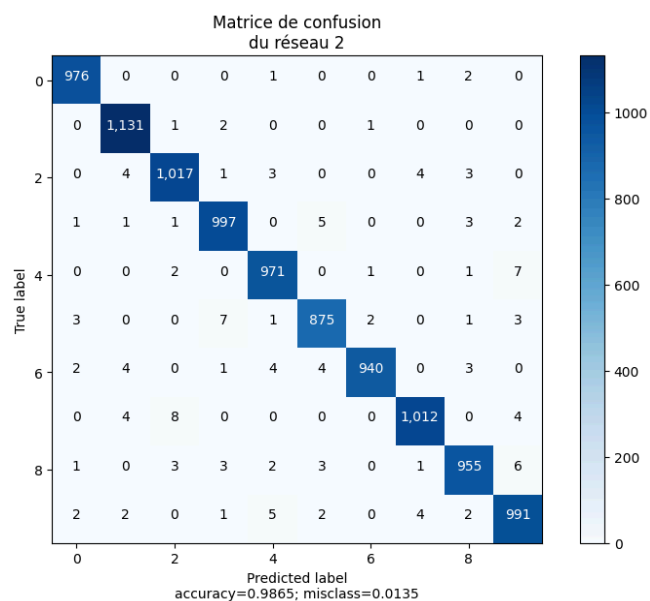
On prend le max de chaque score pour chaque chiffre. Enfin, on effectue la prédiction en prenant le score maximum parmi les 10 valeurs :



Dans ce cas, si un des réseaux surestime grandement le score d'un chiffre, cela peut facilement induire une erreur de prédiction.

4. Analyse des résultats





On voit que vis à vis d'un réseau seul, la combinaison de trois réseaux présente une nette amélioration de la précision. De plus, l'on voit qu'il y a peu de cas où les réseaux sont en total désaccord, ainsi le choix de la fonction de décision n'a qu'un impact minime sur la précision qui est déjà élevée.

En regardant les matrices de confusion des 3 réseaux différents, on voit ce qui fait la force de cette méthode par exemple, le réseau 3 a très tendance (13 fois) à confondre les 2 avec des 7 alors que les 2 autres

réseaux beaucoup moins. Tandis que les réseaux 2 et 3 sont meilleurs à différencier les 9 des 4 que le réseau 1.

Ainsi dans les matrices de confusion où l'on combine 3 réseaux, on voit que le nombre de prédictions fausses s'harmonise ce qui augmente la précision.

De la même manière que dans la partie I, les sources d'erreurs sont le plus souvent les chiffres mal écrits qui sont illisibles.

IV - Conclusion

1. Conclusion sur la meilleure méthode

En analysant la 4ème matrice, on voit que les caractères qui aboutissent le plus souvent à un conflit des trois réseaux sont les 1 et les 8.

La méthode avec une meilleure accuracy est celle où l'on réalise la moyenne de la sortie de chaque réseau sans passer par les occurrences. En effet, elle permet de prendre en compte chacun des scores attribués par chiffre en sortie de chaque réseau qu'uniquement la prédiction de chaque réseau:

```
y_pred1 = [0.1, 0.1, 0, 0.5, 0.5, 0.3, 0.6, 0.1, 0, 0.2]
```

```
y_pred2 = [0.1, 0, 0, 0.3, 0.5, 0.3, 0.6, 0, 0, 0]
```

```
y_pred3 = [0.2, 0.1, 0.3, 0.2, 0.4, 0.3, 0.1, 0.1, 0, 0.2]
```

On voit que la méthode des occurrences donnerait la valeur verte plutôt que la rouge que donnerait la

méthode de la moyenne directe, sur laquelle la plupart des réseaux semble s'accorder.

Le principal problème de cette méthode est si l'un des réseaux donne un score aberrant: cela faussera la moyenne et donc le résultat.

Exemple:

```
y_pred1 = [0.2, 0.1, 0, 0.5, 0.4, 0.3, 0.6, 0.1, 0,0.2]
y_pred2 = [0.2, 0, 0, 0.3, 0.5, 0.3, 0.6, 0, 0,0]
y_pred3 = [1, 0.1, 0.3, 0.2, 0.4, 0.3,0, 0.1, 0,0.2]
```

Ici on voit que l'occurrence aurait choisi la valeur verte plutôt que la valeur rouge que choisit la moyenne et qui pourrait être dû à un réseau mal entraîné.

En fait, cela dépend des variations entre chaque réseau, ce qui revient également à la complexité de notre problème.

Ici, nous avons un problème relativement simple qui permet une forte précision, cela aboutit à des réseaux plutôt similaires malgré les variations de structure et limite le risque de valeur aberrante d'un seul réseau

c'est pourquoi dans notre cas la méthode de la moyenne sans compter les occurrences est la plus fiable ici.

Il convient de noter que l'écart de précision est minime entre la méthode des moyennes et la méthode des occurrences avec la meilleure fonction de décision. Néanmoins, les réseaux étant entraînés et testés sur un grand nombre de données, on peut considérer que ces variations ne sont pas dûes au hasard.

1. Potentiels approfondissements

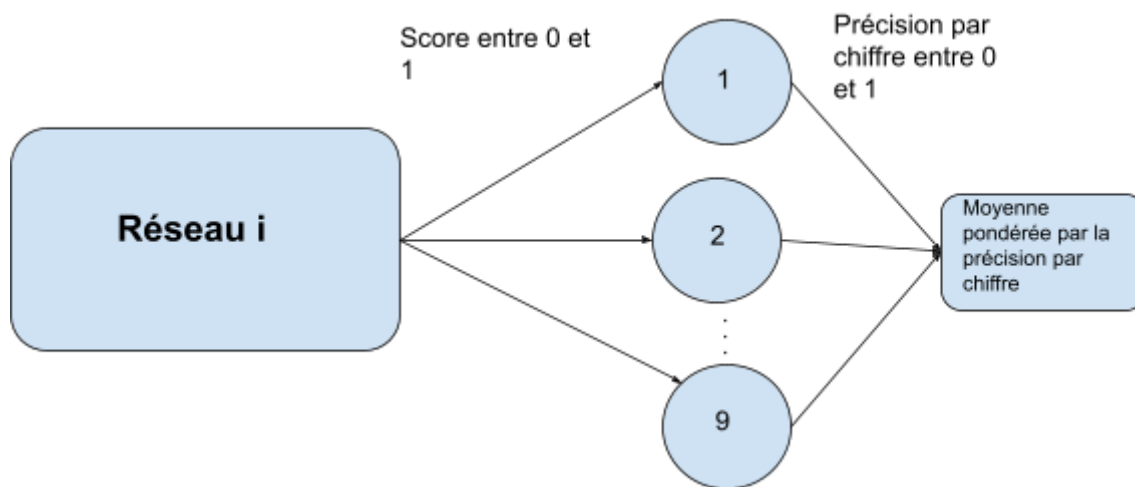
a. Augmenter le nombre de réseaux

L'augmentation du nombre de réseaux de neurones utilisés apparaît comme une solution évidente pour améliorer l'accuracy de notre algorithme. Cependant, l'utilisation de 3 réseaux permet déjà d'obtenir une accuracy à la hauteur de 99%, augmenter le nombre de réseaux n'aurait donc qu'une influence minime sur cette

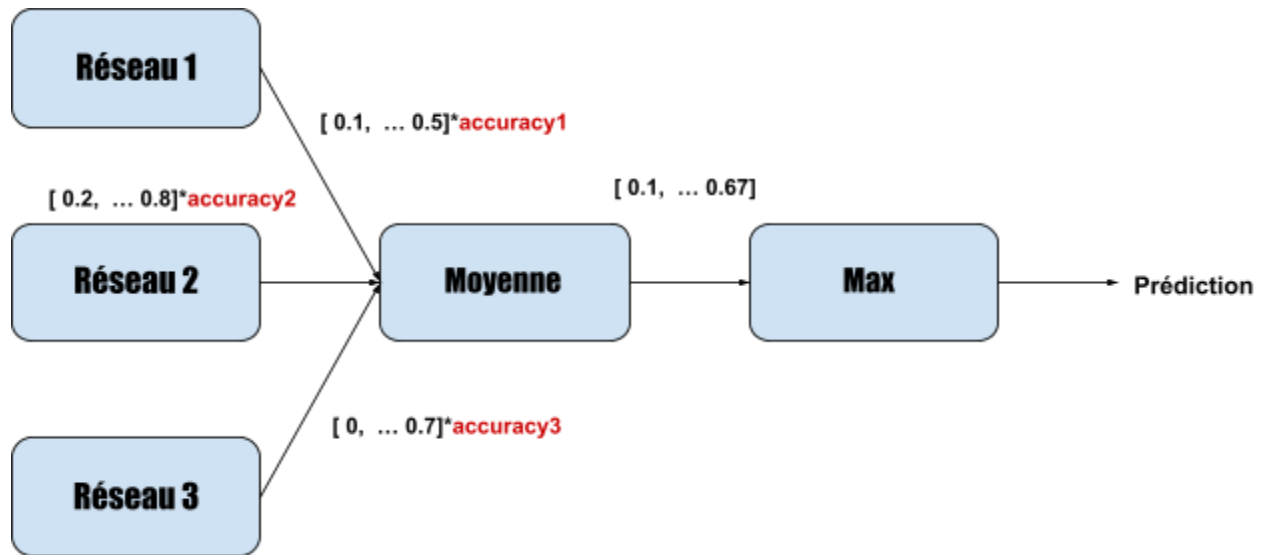
dernière. Cette légère amélioration ne justifie donc pas forcément l'utilisation de plus de 3 réseaux qui augmenterait la complexité de l'algorithme et son temps de calcul.

b. Pondérer chacune des moyennes par la précision de chaque réseau

On reprendrait la méthode des moyennes à la sortie de chaque réseau en pondérant par la précision qui est en quelque sorte l'indice de confiance du réseau ou encore plus précisément par l'accuracy pour chaque chiffre.



Cela devrait limiter les problèmes liés aux valeurs aberrantes dans cette méthode. Voici la structure si l'on pondère par l'accuracy du réseau.



c. Concevoir, entraîner et tester un réseau spécialisé

Souvent le réseau a du mal à différencier certains chiffres, notamment 2 et 7. Dans ce cas là, il peut être intéressant de faire un autre réseau spécialisé dans la différenciation de deux chiffres auquel on fait appel s'il y a désaccord entre les 3 réseaux. L'idéal serait d'en faire un pour différencier chaque chiffre, sachant que l'on a 10 chiffres, cela fait un total de $9+8+7+6+5+4+3+2+1 = 45$

réseaux ce qui paraît beaucoup, d'où la nécessité de faire ces réseaux uniquement quand il y a de nombreuses confusions entre deux chiffres.

On peut également entraîner un réseau à reconnaître si oui ou non ce chiffre est un 9, cela réduirait le nombre de réseau à 10 réseaux supplémentaires.

3. Point de vue personnel

Sur le plan des connaissances, nous avons acquis une compréhension approfondie des mécanismes sous-jacents des réseaux de neurones et de leur application pratique dans la reconnaissance d'images. La complexité des algorithmes et la manière dont ils simulent le fonctionnement du cerveau humain nous ont fascinés et ont élargi notre perspective sur les capacités de l'informatique moderne.

Ce que nous avons particulièrement apprécié dans cet enseignement, c'est la possibilité de combiner théorie et pratique. Le fait de pouvoir expérimenter directement avec des modèles de réseaux de neurones et d'observer leur comportement en temps réel a rendu l'apprentissage plus tangible et significatif pour nous. Cela a été renforcé par la qualité des enseignants, leur pédagogie et leur dynamisme furent de réels moteurs de notre motivation

Annexe

- Lien du drive:
https://drive.google.com/drive/folders/1prL4h-6eyoRGimtO7vITEbclk2NjWVA8?usp=drive_link
- Vidéo: <https://clipchamp.com/watch/VDDPLDkzVeW>